



```
In [3]: #Exp 1b
import nltk
from nltk.corpus import brown
from nltk.tag.hmm import HiddenMarkovModelTrainer
from nltk.probability import ConditionalFreqDist, LidstoneProbDist

nltk.download('brown')
nltk.download('universal_tagset')

tagged_sentences = brown.tagged_sents(tagset='universal')

tagged_sentences = [
    [(word.lower(), tag) for (word, tag) in sent]
    for sent in tagged_sentences
]

trainer = HiddenMarkovModelTrainer()
hmm_model = trainer.train_supervised(
    tagged_sentences,
    estimator=lambda fd, bins: LidstoneProbDist(fd, 0.1, bins)
)

transition_fd = ConditionalFreqDist()
for sent in tagged_sentences:
    tags = [tag for (_, tag) in sent]
    for i in range(len(tags) - 1):
        transition_fd[tags[i]][tags[i + 1]] += 1

emission_fd = ConditionalFreqDist()
for sent in tagged_sentences:
    for word, tag in sent:
        emission_fd[tag][word] += 1

def predict_next_word(sentence):
    words = sentence.lower().split()

    tagged = hmm_model.tag(words)
    last_pos = tagged[-1][1]

    next_pos = transition_fd[last_pos].max()

    if next_pos == 'VERB' and words[-1] in ['will', 'can', 'may', 'should', 'w
        common_base_verbs = ['be', 'have', 'do', 'go', 'make', 'see']
        for verb in common_base_verbs:
            if verb in emission_fd['VERB']:
                return verb, next_pos
```

```
    next_word = emission_fd[next_pos].max()
    return next_word, next_pos

test_sentence = "the president will"
word, pos = predict_next_word(test_sentence)

print("Input sentence:", test_sentence)
print("Predicted next word:", word)
print("Predicted POS:", pos)
```

```
[nltk_data] Downloading package brown to /root/nltk_data...
[nltk_data]   Package brown is already up-to-date!
[nltk_data] Downloading package universal_tagset to /root/nltk_data...
[nltk_data]   Package universal_tagset is already up-to-date!
```

```
Input sentence: the president will
Predicted next word: be
Predicted POS: VERB
```

In []: